

Fascicule du Cours :
Algorithmique et Structures de Données

Plan du cours

Chapitre I – Rappel et conventions syntaxiques

- 1. Définition**
- 2. Structure d'un algorithme**
- 3. Types de données**
- 4. Opérations de base**
- 5. Algorithmique et programmation**
- 6. Structures conditionnelles**
- 7. Structures itératives**
- 8. Décomposition modulaire**
 - a. Procédures
 - b. Fonctions
 - c. Transmission de paramètres

Chapitre II - Les objets structurés

- 1. Tableaux**
 - a. Tableaux à une dimension
 - b. Tableaux à deux dimensions
- 2. Chaînes de caractères**
- 3. Enregistrements**

Chapitre III – La récursivité

- 1. Définition et exemples**
- 2. Propriétés des algorithmes récursifs**
- 3. Notion de pile d'exécution**

Chapitre IV – Allocation dynamique de la mémoire

- 1. Notion de pointeur**
- 2. Déclaration d'un pointeur**
- 3. Réservation de la mémoire**
- 4. Accès à la variable pointée**
- 5. Libération de la mémoire**

Chapitre V – Les fichiers

- 1. Intérêt des fichiers**
- 2. Organisation**
- 3. Types d'accès**
- 4. Déclaration**
- 4. Instructions**
 - a. Ouverture d'un fichier
 - b. Lecture depuis un fichier
 - c. Ecriture dans fichier
 - d. Fermeture d'un fichier

Chapitre VI – Types abstraits de données

- 1. Définition d'un TAD**
- 2. Signature d'un TAD**
- 3. Description des propriétés d'un TAD**

Chapitre VII – Listes

- 1. Définition des listes**
- 2. Type abstrait "Liste"**
- 3. Représentation des listes**
 - Représentation contigüe
 - Représentation chaînée
- 4. Autres types de listes**
 - Liste circulaire
 - Liste doublement chaînée

Chapitre VIII - Piles

- 1. Définition**
- 2. Type abstrait « Pile »**
- 3. Représentation des Piles**
 - Représentation contigüe
 - Représentation chaînée

Chapitre IX – Files

- 1. Définition**
- 2. Type abstrait de données « File »**
- 3. Représentation des files**
 - Représentation contigüe
 - Représentation chaînée

Chapitre X - Arbres binaires

- 1. Définition**
- 2. Terminologie**
- 3. Définition du TAD**
- 4. Mesures sur les arbres**
- 5. Arbres binaires particuliers**
- 6. Représentation des arbres binaires**
 - Représentation chaînée
 - Représentation contigüe
- 7. Parcours d'un arbre binaire**
 - Parcours en largeur
 - Parcours en profondeur main gauche

Chapitre XI - Arbres binaires particuliers

- 1. Arbres binaires de recherche**
 - Recherche d'un élément
 - Adjonction d'un élément aux feuilles
 - Adjonction d'un élément à la racine

- d. Suppression d'un élément
- 2. Arbres binaires équilibrés**
 - a. Ajout d'un élément
 - b. Suppression d'un élément
 - c. Algorithmes de rééquilibrage

Chapitre XII – Graphes

- 1. Définitions et exemples**
- 2. Terminologie**
- 3. Représentations des graphes**
 - a. Utilisation de matrice
 - b. Utilisation de listes d'adjacence
- 4. Parcours de graphes**
 - a. Parcours en profondeur
 - b. Parcours en largeur

Références bibliographiques

MALGOUYRES, Rémy, ZROUR, Rita, et FESCHET, Fabien. *Initiation à l'algorithmique et à la programmation en C-3e éd. : Cours avec 129 exercices corrigés*. Dunod, 2014.

CORMEN, Thomas H., LEISERSON, Charles Eric, RIVEST, Ronald L., et al. *Introduction à l'algorithmique*. Dunod, 1997.

COURTIN, Jacques, KOWARSKI, Irène. *Initiation à l'algorithmique et aux structures de données Volume 2*. Dunod, 1997.

COURTIN, Jacques, KOWARSKI, Irène. *Initiation à l'algorithmique et aux structures de données Volume 1*. Dunod, 1994.

FROIDEVAUX, Christine, GAUDEL, Marie-Claude, et SORIA, Michele. *Types de données et algorithmes*. Ediscience International, 1993.

BEN RHOUMA, Kamel. *Cours Algorithmique et Structures de données*. ENSI, 2007.

CHAPITRE I – Rappel et conventions syntaxiques

1. Définition

L'origine du mot *algorithme* découle du nom latinisé du mathématicien perse du 9^{ème} siècle « Abu Ja'far Mohammed Ibn Mûsâ Al-Khowâ-rismî ».

Le terme algorithme admet, depuis longtemps, plusieurs définitions qui convergent. Froidevaux et al. définissent ce concept comme étant [Froidevaux et al., 1993] :

« La composition d'un **ensemble fini d'étapes**, chaque étape étant formée d'un nombre **fini d'opérations** dont chacune est :

- Définie de façon **rigoureuse et non ambiguë** ;
- **Effective** c'est-à-dire pouvant être réalisée par une machine. »

Tout algorithme est donc caractérisé par [Courtin et al., 1994] :

- Un ensemble d'opérations à exécuter.
- Un ordre d'exécution de ces différentes opérations dicté par la logique d'enchaînement et conditionné par les structures mises en œuvre.
- Un début et une fin.

Le domaine qui étudie les *algorithmes* est désigné par : « algorithmique ».

2. Structure d'un algorithme

Algorithme <nom de l'algorithme>

CONST

<nom_de_la_constant = valeur>

TYPE

<liste des structures>

VAR

<nom-variable> : <type-variable>

FONC

<liste des fonctions >

PROC

<liste des procédures>

DEBUT

<corps de l'algorithme : ensemble des instructions – une instruction par ligne>

FIN

Dans ce qui suit, nous présentons un exemple d'algorithme :

Supposons qu'on veut calculer la moyenne des notes d'un élève dans une matière donnée et qu'on dispose de 3 notes : N1, N2 et N3.

Algorithme Moyenne**CONST**

c1=2

c2=3

c3=4

VAR

Moyenne, N1, N2, N3 : réel

DEBUT

Ecrire(" Donnez les trois notes dans l'ordre")

Lire (N1,N2,N3)

Moyenne $\leftarrow (N1*c1+N2*c2+N3*c3)/(c1+c2+c3)$

Ecrire (" La moyenne est : ", Moyenne)

FIN**3. Types de données**

Dès que l'on a besoin de stocker une information au cours d'un algorithme, on utilise une **variable**.

a. Types de base**i. Booléen**

Valeurs : Vrai, Faux.

Opérations

- * Opérations relationnelles (Faux < Vrai)
- * Opérations logiques : négation (non), conjonction (et), disjonction (ou)

A	B	Non A	A et B	A ou B
V	V	F	V	V
V	F	F	F	V
F	V	V	F	V
F	F	V	F	F

ii. Caractère

Valeurs : c'est un ensemble fini de symboles totalement ordonnés. Un code ASCII est associé à chaque caractère.

Opérations

- * Opérations relationnelles par exemple : 'a' < 'b' ; 'a' > 'A'
- * Opérations : ORD, CHR

iii. Entier

Valeurs : c est un sous ensemble de $\mathbb{Z}$. Il s'agit d'une représentation en machine des entiers relatifs.

Opérations

- * Opérations relationnelles : $= ; < ; > ; \leq ; \geq ; < >$
- * Opérations arithmétiques : $+, -, *, /$
- * DIV : division entière
- * MOD : reste de la division entière
- * PUISSANCE

iv. Réel

Valeurs : un sous ensemble de $\mathbb{R}$. Il s'agit d'une représentation en machine des réels.

Opérations

- * Opérations relationnelles : $= ; < ; > ; \leq ; \geq ; < >$
- * Opérations arithmétiques : $+, -, *, /$
- * Puissance
- * Autres opérations comme $\cos, \sin \dots$

Outre les types définis ci-dessus ; il est possible de définir de nouveaux types comme suit.

b. Type scalaire énuméré

Le type scalaire énuméré permet de représenter un ensemble ordonné et fini de valeurs désignées par des identificateurs.

Définition et déclaration

Type `nom_type` = (`val1, ..., valk`)

Une variable de type énuméré prendra une des valeurs figurant entre parenthèses.

Exemple : Type `t_couleur` = (blanc, bleu, rouge)

Var

`cl : t_couleur`

`cl ← blanc`

Opérations

- * Opérations relationnelles
- * `Ord()` : rang dans le domaine.
- * `Succ()` : successeur dans le domaine
- * `Pred()` : prédécesseur dans le domaine
- * `Inc(v,n)` : incrémenter v de n

Exemples : `ord(bleu)=1`, `succ(bleu)=rouge`, `pred(bleu)= blanc`.

c. Type intervalle

Le type intervalle permet de restreindre le groupe de valeurs d'un type scalaire discret (entier, booléen, caractère ou type énuméré).

Définition du type :

Type `nom_intervalle = borne_inf...borne_sup`

Avec `borne_inf < borne_sup`.

Les opérations que nous pouvons effectuer sont celles du type des variables contenues dans l'intervalle.

Exp : Type `T_chiffre=0..9`

`T_majuscule='A'..'Z'`

4. Opérations de base

Dans un schéma séquentiel, on distingue deux types d'instructions :

- Des instructions simples, élémentaires permettant de faire un traitement ;
- Des instructions permettant de commander le déroulement de l'exécution, appelées structures de contrôle.

a. Affectation

Le rôle de cette instruction est de placer une valeur dans une variable (associer une valeur à une variable).

`<nom variable> ← <expression>`

`<expression>` : constante | variable | expression

La variable et l'expression doivent être de même type ou de types compatibles (Exp : il est possible d'affecter un entier à une variable réelle).

Exemple

Test()

var

`i,j : entier`

Debut

`i ← 10`

`j ← 20`

`i ← 2*j`

Fin**b. Lecture**

L'utilisateur introduit des valeurs qui seront affectées à des variables réservées par le programmeur.

L'utilisateur doit respecter le type des éléments à introduire.

Lire (`< liste variables >`)

< liste variables > : <variable1>, ... <variable n>

Exemple

Var

i : entier

r : réel

Debut

Lire(i,j)

Fin

c. Ecriture

Cette instruction permet aussi bien l’affichage d’un texte que l’affichage des valeurs d’une liste de variables (une ou plusieurs).

Ecrire (<texte>)

Exemple

Var

i, j : entier

Debut

Ecrire (“Bonjour”)

lire (i, j)

i ← i*j

Ecrire (i)

Fin

Remarque

On peut regrouper dans la syntaxe de l’instruction Ecrire, la liste des variables et le texte. Par exemple : Ecrire ("La valeur de i est", i, " la valeur de j est ", j)

5. Algorithmique et programmation

La mise en œuvre d’un algorithme consiste en l’écriture des opérations qui le composent dans un langage de programmation et constitue alors la brique de base d’un programme informatique. Un algorithme exprime les instructions permettant de résoudre un problème donné indépendamment des langages de programmation. Lors du passage de l’algorithmique à la programmation, le travail revient à résoudre des problèmes de syntaxe, ou de types d’instructions propres à ce langage.

Afin d’écrire un algorithme, on se base généralement sur une série de conventions désignée par « **pseudo-code** », qui est comparable à un langage de programmation dont on aurait évacué la plupart des problèmes de syntaxe. Puisqu’il est purement conventionnel, ce pseudo-code peut varier légèrement d’une référence à une autre.

Pour être compréhensible, un algorithme doit être présenté de manière lisible ; il ne doit pas être trop long, ce qui nécessite parfois de le décomposer en modules. Certaines formes de programmation telles que les branchements sont à éviter ; par contre la récursivité permet une expression concise des algorithmes et facilite leur analyse.

6. Structures conditionnelles

a. Schéma conditionnel à simple choix (Si – Alors – Fsi)

i. Syntaxe

Si <condition> alors

 <Traitement>

Fsi

<condition> : expression à résultat booléen

<traitement> : suite d'instructions

Fsi : Fin si

ii. Sémantique

Si <condition> est vraie alors on exécute le bloc <traitement> et on continue en séquence à l'instruction qui suit <Fsi>.

Si <condition> est fausse, on continue en séquence avec l'instruction qui suit immédiatement <Fsi> .

b. Schéma conditionnel à double choix (Si – Alors – Sinon - Fsi)

i. Syntaxe

Si <condition> alors

 <Traitement 1>

Sinon

 <Traitement 2>

Fsi

ii. Sémantique

Si <condition> est vraie alors on exécute le bloc <traitement 1> et on continue en séquence à l'instruction qui suit <Fsi>.

Si <condition> est fausse, on exécute les instructions du bloc <traitement 2> et on passe à l'instruction qui suit immédiatement <Fsi>.

c. Schéma conditionnel à choix multiple

i. Syntaxe

Selon <v>

 <v1> : <Traitement 1>

 <v2> : < Traitement 2>

 <vn> : < Traitement n>

Sinon : <Traitement >

Fin selon

ii. Sémantique

<Traitement > : suite d'instructions

<v> : identificateur d'une variable de type scalaire.

$\langle v_i \rangle$: une valeur ou une liste composée d'une suite de valeurs (**Exp** : 2, 5, 8) ou d'une fourchette (**Exp** : 0..9). Les valeurs contenues dans les $\langle v_i \rangle$ doivent être de même type que $\langle v \rangle$.

Lorsque $\langle v \rangle = \langle v_i \rangle$ on exécute le traitement correspondant et on passe à l'instruction qui suit immédiatement **Finselon**.

Le bloc **<sinon>** est facultatif.

Remarque

L'instruction selon peut être ramenée à un enchaînement de « Si » imbriquées.

Exemple d'utilisation

Exp()

Var

k, x : entier

Debut

x $\leftarrow$ 0

lire(k)

Selon k

5 : x $\leftarrow$ x+6

6..10 : x $\leftarrow$ x+18

11, 12, 13 : x $\leftarrow$ x-5

Sinon

x $\leftarrow$ 1

Fin selon

Fin

7. Structures itératives

Les structures itératives sont des instructions qui permettent de répéter l'exécution d'une même séquence d'instructions un nombre fini de fois.

On distingue deux types de répétitions :

- * L'itération : quand on connaît le nombre de fois que l'on va répéter une instruction ou une séquence d'instructions.
- * La répétition conditionnelle : quand la poursuite d'une répétition dépend d'une condition à résultat booléen.

a. Boucle « Pour »

Pour $\langle \text{compteur} \rangle$ **de** $\langle \text{valeur initiale} \rangle$ **à** $\langle \text{valeur finale} \rangle$ **pas** $\langle \text{incrément} \rangle$ **faire**

$\langle \text{Traitement} \rangle$

Fpour

$\langle \text{compteur} \rangle$: l'identificateur d'une variable de type scalaire.

$\langle \text{valeur initiale} \rangle$, $\langle \text{valeur finale} \rangle$, $\langle \text{incrément} \rangle$: une variable de même type que le compteur ou une constante de même type que le compteur ou une expression dont le résultat est de même type que le compteur.

$\langle \text{Traitement} \rangle$: une ou plusieurs instructions quelconques.

Exemple :**var**

i, j, n : entier

Debut

n ← 5

j ← 1

Pour i de j à n pas (2) faire

Ecrire(i)

Fpour**Fin****Remarques**

Il est interdit de modifier : <compteur>, <valeur initiale>, <valeur finale> et <incrément>.

- * <valeur initiale> <= <valeur finale> Ssi <incrément> >0
- * <valeur initiale> >= <valeur finale> Ssi <incrément> <0
- * <incrément> ne doit jamais être nul.

Lorsque la valeur de <incrément> n'est pas précisée elle est considérée comme étant égale à 1.

L'instruction Pour est utilisée lorsque le nombre d'itérations est connu.

b. Boucle « Tant que »**Tant que** <condition> **faire**

<Traitement>

Fin tant que

<condition> : expression à résultat booléen

<Traitement> : un ensemble d'instructions telles qu'il existe au moins une instruction modifiant la condition.

Exercice

Ecrire un algorithme permettant de saisir une suite d'entiers positifs ou nuls se terminant par une valeur négative, de faire la somme de ces entiers et de l'afficher.

c. Boucle « Répéter »**Répéter**

<Traitement>

Jusqu'à <condition>**Remarque**

Cette instruction ne peut être utilisée que si on est sûrs que le traitement est exécuté **au moins une fois**. La boucle précédente peut être remplacée par la boucle suivante :

Tant que non <condition> **faire**

<Traitement>

FTQ

8. La construction modulaire d'algorithmes

Lors de la conception d'un programme résolvant un problème général, il est nécessaire de décomposer le problème en différents sous-problèmes moins complexes à résoudre.

Ces différents sous-problèmes peuvent être résolus grâce à des modules ou sous-programmes.

L'utilisation des sous-programmes présente de nombreux avantages. Nous citons :

- La possibilité de réutilisation ;
- La facilité de la résolution de problèmes en ne s'intéressant qu'à l'écriture d'un module à la fois ;
- La factorisation du code : les sous-programmes permettent de ne pas répéter plusieurs fois une même séquence d'instructions au sein de l'algorithme.

En algorithmique, il existe deux types de sous-programmes :

- Les procédures
- Les fonctions

Lors de la conception d'un programme deux aspects apparaissent :

- **La définition de la procédure ou de la fonction ;**
- **L'appel de la procédure ou de la fonction dans le programme.**

Une fois explicité (l'algorithme défini), un sous programme peut être utilisé dans d'autres programmes ou sous-programmes. Le (sous-) programme qui utilise un sous-programme est désigné par (sous-) programme **appelant**.

a. Procédures

i. Déclaration

Procédure <Nom procédure> (<Liste-paramètres>)

var

<variables-locales>

Début

<Traitement>

Fin

<Nom procédure> : identificateur

<Liste-paramètres> : <par1> : <Type1>, <par2> : <Type2>,, <par n> : <Type n>

<par i> : identificateurs de variables

<variables-locales> : <var1> : <Type1>

<var2> : <Type2>

Une variable définie dans un sous-programme est appelée **variable locale**. La portée d'une variable locale est uniquement le sous-programme qui la déclare.

<Traitement> : liste des instructions de la procédure

ii. Appel de procédure

Un appel de procédure se fait de la façon suivante :

<Nom procédure> (<Liste-paramètres>)

<Liste-paramètres> : <par1>, <par2>,.....,<par n>

<par i> =identificateurs de variables ou constante ou expression.

iii. Exemple de procédure

Algo Exp1

Var

A, B : entier

Procédure **AfficheMax**(X :entier, Y :entier)

Début

si (X>Y)
 alors écrire(X)
 sinon
 écrire (Y)
 fsi

Fin

Début (Algorithme principal : algorithme appelant)

 Ecrire(« Donnez deux entiers »)

 Lire(A,B)

 AfficheMax(A,B)

Fin

b. Fonctions**i. Déclaration**

Fonction <Nom fonction> (<Liste-paramètres>) : <type>

var

<variables-locales>

Début

 <Traitement>

 <Nom fonction> ← <variable>

Fin

<Nom fonction> : identificateur

<Liste-paramètres> : <par1> : <Type1>, <par2> :<Type2>,.....,<par n> : <Type n>

<par i> : identificateurs de variables

<type> : type de retour de la fonction

<variables-locales> : <vari> : <Typei>

<Traitement> : liste des instructions de la fonction

<variable> : variable de même type que la fonction ou de type compatible.

ii. Appel de fonction

Une fonction est utilisée dans une instruction :

- Dans la partie droite d'une affectation ($y \leftarrow f(x)$)
- Dans une expression (arithmétique ou logique selon le type de la fonction) ($y \leftarrow f(x)+2$)
- Comme paramètre d'une procédure ou d'une fonction $P(f(x))$, $f(f(x))$.

iii. Exemple de fonction

Algo Max

Var

A, B, C : entier

Fonction **Max**(X, Y : entier) : entier

Var

W : entier

Début

Si (X>Y) alors

 W $\leftarrow$ X

Sinon

 W $\leftarrow$ Y

Fsi

 Max $\leftarrow$ W

Fin

Début (Algorithme principal : algorithme appelant)

Ecrire (« Donner deux entiers »)

Lire (A, B)

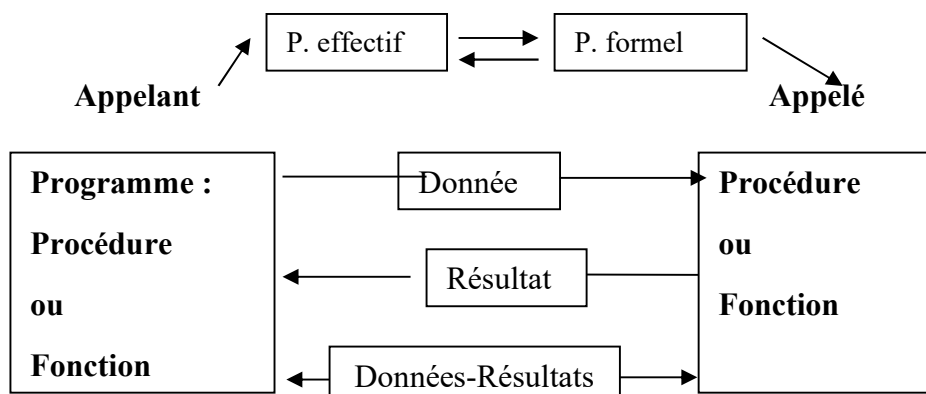
C $\leftarrow$ Max(A, B)

Ecrire (« Le maximum est : », C)

Fin

c. Transmission des données

i. Les types de paramètres



Lors de l'utilisation des sous-programmes, une transmission de données et de résultats est mise en jeu. On distingue :

- **Les paramètres formels** : utilisés dans la **définition** de la fonction ou procédure. Ce sont des variables qui peuvent prendre plusieurs valeurs.
- **Les paramètres effectifs** : ce sont les paramètres utilisés de façon effective dans un appel de fonction ou de procédure (*ce sont les valeurs qu'on utilisera pour effectuer les calculs dans la fonction ou procédure*)

Le nombre de paramètres **effectifs** doit être égal au nombre de paramètres **formels**.

Le paramètre **effectif** doit avoir **le même type ou un type compatible** avec le paramètre **formel** correspondant.

La correspondance entre paramètres **effectifs** et paramètres **formels** se fait dans l'ordre de leur apparition dans les deux listes.

ii. Les modes de passage de paramètres

Il existe 3 modes de passage de paramètres :

- **Donnée** : dans ce mode de passage, le paramètre doit porter une valeur significative avant l'exécution, il est nécessaire aux calculs effectués dans le sous-programme mais il ne sera pas modifié par celui-ci. C'est le seul mode de passage de paramètre qui admet l'utilisation d'une constante. Ce mode de passage est exprimé par le mot clé : **Don**
- **Résultat** : dans ce mode de passage, le paramètre est destiné à recevoir un résultat. Les instructions du sous-programme affectent obligatoirement une valeur à ce paramètre qui n'a pas nécessairement une valeur significative avant l'exécution. Par contre, après l'exécution du sous-programme, il doit comporter une valeur significative. Ce mode de passage de paramètre est exprimé par le mot clé : **Res**
- **Donnée-résultat** : dans ce mode de passage, le paramètre est utilisé dans les calculs faits dans le sous-programme. Il est en même temps utilisé pour recevoir un résultat. Il doit donc avoir une valeur significative avant et après exécution. Ce mode de passage de paramètre est exprimé par le mot clé : **DonRes**

Exemples :

1/

Procédure **AfficheMax** (X :entier, Y :entier)

Utilisation : AfficheMax (A,B)

Les paramètres **formels** sont : X et Y.

Les paramètres **effectifs** sont : A et B.

X et Y sont des paramètres données. Donc on déclarera la procédure AfficheMax comme suit :

Procédure **AfficheMax** (**Don** X , Y :entier)

2/

Fonction **Max(X, Y : entier) : entier**

Utilisation : C $\longleftarrow$ Max(A, B)

Les paramètres **formels** sont : X et Y.

Les paramètres **effectifs** sont : A et B.

X et Y sont des paramètres données. Donc on déclarera la fonction **Max comme suit :**

Fonction Max(Don X, Y : entier) : entier

3/

Soit la procédure **Permute** qui permute deux entiers X et Y.

Procédure **Permute** (DonRes X, Y :entier)

CHAPITRE II – LES OBJETS STRUCTURÉS

Partie I - Les tableaux

Un tableau est une structure de données homogène contenant des éléments de même type.

1. Tableaux à une dimension

a. Définition de type

<Nom type> = tableau [$b_i..b_s$] de <type de base>

<Nom type> : identificateur associé au tableau

$b_i..b_s$: intervalle sur N qui fixe les indices des éléments du tableau (avec $b_i \leq b_s$).

b_i : borne inférieure, b_s : borne supérieure

<type de base> : le type des éléments du tableau.

Exemple :

b. Déclaration de variables

<identificateur> : <type tableau>

<type tableau> : <Nom type> ou tableau [$b_i..b_s$] de <type de base>

Exemple :

c. Opérations

* *Accès aux éléments d'un tableau*

<Nom tableau> [<indice>]

Où **<Nom tableau>** est l'identificateur du tableau et **<indice>** est la position de l'élément dans le tableau.

Exemple :

T[1] désigne l'élément du tableau à la position 1.

* *Les opérations autorisées sont toutes les opérations autorisées sur le type des éléments (type de base).*

Exemple :

T : tableau [1..15] de entier

T[5] ← T[20]+3 faux

T[6] ← '4' faux

T[10] ← 2* T[3] correct

d. Exercice

Ecrire un algorithme qui applique le principe de la recherche dichotomique pour chercher un élément x dans un tableau trié de N éléments.

2. Tableaux à deux dimensions**a. Définition de type**

<Nom-type> = tableau [<int-ligne>, <int-colonne>] de <type de base>

<Nom-type> : identificateur associé au tableau

<int-ligne> : intervalle de lignes

<int-colonne> : intervalle de colonnes

<type de base> : le type des éléments du tableau

Exemple :

MatriceEntier = tableau [1..10, 1..15] de entier

b. Déclaration de variables

<identificateur> : <type -tableau>

<type-tableau> : <Nom-type> ou tableau [<int-ligne>, <int-colonne>] de <type de base>

Exemple :

MatriceEntier=tableau [1..10,1..15] de entier (Nom-type)

T : MatriceEntier

Ou

T : tableau[1..10, 1..15] de entier

c. Opérations

Accès aux éléments de la matrice

<identificateur-tableau> [<indice-ligne>, <indice-colonne>]

Exemple :

T[1,15] : l'élément se trouvant à l'intersection de la ligne 1 et la colonne 15.

T[i,j] : l'élément se trouvant à l'intersection de la ligne i et la colonne j.

Partie II - Les chaînes de caractères**1. Déclaration, initialisation et accès aux éléments**

Le type chaîne permet de décrire des objets formés par une succession de N caractères avec N compris entre 0 et 255. Si N=0 on dit que la chaîne est vide.

Dans la plupart des langages de programmation, il existe des "outils" pour manipuler les chaînes de caractères. Au niveau des algorithmes, nous introduisons quelques fonctions prédéfinies qui correspondent aux "outils" classiquement fournis par les langages. Cette liste partielle peut bien sûr être complétée en fonction de vos besoins.

❖ **Déclaration d'une chaîne de caractères**

Comme indiqué dans la définition précédente, une chaîne de caractères est composée d'au plus 255 caractères donc il n'est pas nécessaire (mais il est possible) de faire figurer dans la déclaration d'une chaîne le nombre maximum de caractères qu'elle peut contenir.

Exp :

Nom : Chaîne [10] *//la chaîne nom peut contenir au maximum 10 caractères*

Prenom : Chaîne *// la chaîne nom peut contenir au maximum 255 caractères*

❖ **Accès à un élément de la chaîne**

Ch ← 'Bonjour'

Ch[1]='B' Ch[4]='j'

2. Lecture et écriture d'une chaîne de caractères

❖ **Lecture**

Ch : chaîne

Lire (ch) *//la chaîne est manipulée comme une entité*

❖ **Écriture**

Ch : chaîne

Ecrire (ch)

3. Fonctions de base sur les chaînes de caractères

❖ **Fonction longueur (ch : chaîne) : entier**

Retourne la longueur de la chaîne *ch* c'est-à-dire le nombre de caractères dont elle est constituée.

Exp : ch ← 'Bonjour'

Longueur (ch)=7

❖ **Fonction concat (ch1:chaîne, ch2:chaîne) : chaîne**

Retourne une chaîne de caractères formée par la concaténation de *ch1* et de *ch2*. La chaîne résultat est formée de *ch1* suivi de *ch2*.

Exp : ch1 ← 'Bon', ch2 ← 'jour', ch3 ← concat(ch1,ch2)

Ch3 contient 'Bonjour'

❖ Fonction sous_chaine (ch:chaîne, i:entier, l:entier) : chaîne

Retourne une sous-chaîne de longueur *l* extraite de la chaîne *ch*, à partir de la position *i*.

Exp : sousChaine("informatique", 6, 2) retourne la chaîne "ma".

❖ Fonction Pos (ch1:chaîne, ch2:chaîne) : entier

Retourne la première position de la chaîne *ch1* dans la chaîne *ch2*. Si *ch1* n'est pas dans *ch2*, elle retourne 0. **Exp :** Pos ('jour', 'Bonjour') donne 4

❖ Procédure Efface (ch : chaîne, p : entier, n :entier)

Efface *n* caractères de *ch* à partir de la position *p*.

Exp : Efface ('CD ROM',3 ,4) la chaine devient 'CD'.

❖ Procédure Insère (ch1 : chaîne, ch2 : chaîne, p : entier)

Insère la chaîne *ch1* dans la chaîne *ch2* à partir de la position *p*. Le caractère numéro *p* et les suivants sont décalés vers la droite.

Exp : *ch1* ← 'DE', *ch2* ← 'ABCFG'

Insère (*ch1*, *ch2*, 4) *ch2* devient 'ABCDEFGF'

❖ Procédure Convch (d : reel, ch : chaîne)

Convertit un nombre en une chaîne de caractères et l'affecte à *ch*.

Exp : Convch (2007, *ch*) *ch* contient la chaîne '2007'

❖ Procédure Valeur (ch : chaîne, d : reel, erreur : entier)

Convertit une chaîne *ch* en une valeur numérique décimale et l'affecte à la variable *d*. Le paramètre *erreur* contient 0 si la conversion s'est déroulée sans erreur, sinon elle contiendra le numéro du caractère qui a déclenché l'erreur.

Exp :

Ch ← '10.5'

Valeur (*ch*,*d*,*e*) *d* contient le nombre 10.5 et *e* contient 0

Ch ← '01/01/2008'

Valeur (*ch*,*d*,*e*) *d* contient 0 et *e* contient 3 (le caractère '/' n'est pas un chiffre).

Partie III - Les enregistrements

1. Définition

Un enregistrement ou article est une structure de données composée d'un nombre fixe d'éléments pouvant être de types différents. Il permet de regrouper dans un même type un ensemble d'informations caractérisant un objet donné. Les éléments d'un article sont appelés « champs » et peuvent être à leur tour des structures de données.

2. Syntaxe

a. Déclaration de type

<Nom-type> : identificateur associé à l'article

<Champ i> : identificateur du ième champ

<type i> : le type du ième champ

Exemples

b. Déclaration des variables

<id-var> : <Nom type article>

Exemple : R1,R2 : Rectangle

P1 : Personne

c. Opération d'accès

<id-var>. <nom-champ>

Exemple :

R1.largeur ← 3

R1.longueur $\leftarrow$ 5

P1.nom $\leftarrow$ « Mohamed »

P1.age $\leftarrow$ 10

3. Exercice

Ecrire un algorithme qui lit une suite d'informations concernant 30 étudiants et les stocke dans une structure de votre choix.

Chaque étudiant est caractérisé par un numéro de cin, un nom, une date de naissance (jour, mois, année) et un niveau d'études (1, 2 ou 3).

Ecrire un algorithme permettant de :

- a. Calculer le pourcentage des étudiants au niveau 1.
- b. Afficher la liste des noms des étudiants dont l'année de naissance $\geq$ 1990.
- c. Afficher toutes les informations concernant un étudiant dont le numéro de cin est donné.

CHAPITRE III – LA RECURSIVITE

1. Définition et exemples

La récursivité est la possibilité de faire figurer dans la définition d'un objet une référence à l'objet lui-même. Une notation est récursive si elle est partiellement définie à partir d'elle-même.

Il existe deux types de récursivités :

- **La récursivité des objets** : la définition d'un objet est récursive si l'objet apparaît dans cette définition.

Exemple : une chaîne de caractères est soit la chaîne vide soit un caractère suivi d'une chaîne de caractères.

- **La récursivité des traitements** : un sous-programme est dit récursif s'il fait appel à lui-même pour traiter un problème. Dans ce cas, le problème en question est réduit à un problème plus simple mais de même nature.

Un problème se prête bien à la récursivité lorsqu'il peut se décomposer en sous problèmes de même type.

Exemple : La fonction Factorielle

$$n! = n * (n-1) \quad 0! = 1$$

2. Propriétés des algorithmes récursifs

Tout module récursif doit contenir une clause conditionnelle telle que l'évaluation puisse dans certains cas se faire sans récursivité.

Module A()

Debut

Si B alors C
Sinon f(A())
Fsi

Fin

B : est la condition d'arrêt.

C : cas particulier de la récursivité

F(A()) : une fonction de A() qui doit permettre le changement de la valeur de la condition B.

3. Pile d'exécution

La Pile d'exécution (call stack) d'un programme en cours d'exécution, est un emplacement mémoire destiné à mémoriser les paramètres, les variables locales ainsi que l'adresse de retour de chaque module en cours d'exécution. Elle fonctionne selon le principe LIFO (Last-In-First-Out) : dernier entré premier sorti.

Comme la pile a une taille fixe, une mauvaise utilisation de la récursivité peut entraîner un débordement (stack overflow).

Considérons l'exemple de module récursif suivant :

```
Procédure Affiche(n : entier)
```

```
Debut
```

```
Si n>0
```

```
    Affiche (n-1)
```

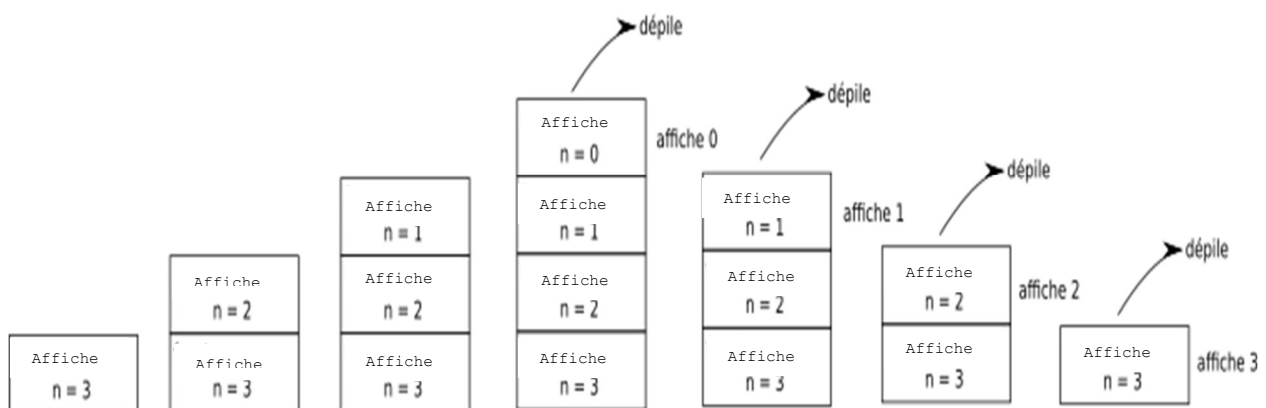
```
    écrire(n)
```

```
Fsi
```

```
Fin
```

- 1er appel de Affiche avec le paramètre $n = 3$; $n > 0$ donc appel de Affiche avec le paramètre $n = 2$
- 2e appel de Affiche avec le paramètre $n = 2$; $n > 0$ donc appel de Affiche avec le paramètre $n = 1$
- 3e appel de Affiche avec le paramètre $n = 1$; $n > 0$ donc appel de Affiche avec le paramètre $n = 0$
- 4e appel de Affiche avec le paramètre $n = 0$; $n = 0$ donc on exécute l'instruction écrire (n)
=> affichage : 0
- on "dépile" (3e appel, $n = 1$) : on exécute l'instruction écrire (n) => affichage : 1
- on "dépile" (2e appel, $n = 2$) : on exécute l'instruction écrire (n) => affichage : 2
- on "dépile" (1er appel, $n = 3$) : on exécute l'instruction écrire (n) => affichage : 3

Le schéma suivant explique le processus en termes de pile d'exécution :



4. Exercices d'application

- Ecrire la version récursive de la fonction Factorielle.
- Ecrire une fonction récursive qui, étant donné un entier X positif et Y un entier ≥ 0 , permet de calculer X^Y .

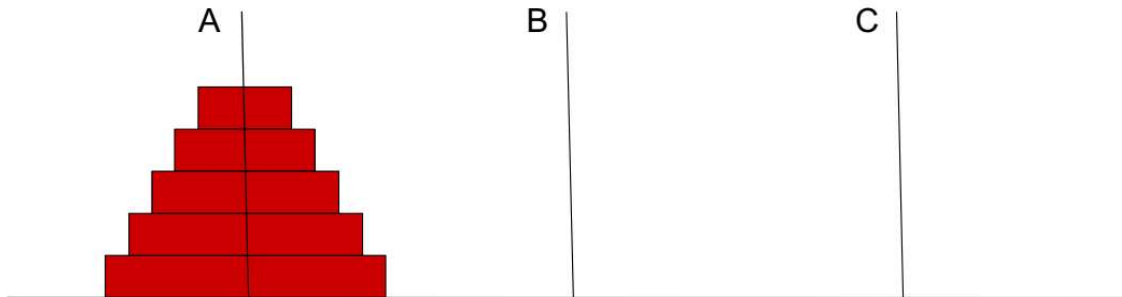
5. Problème des tours de Hanoi

Les tours de Hanoi est un jeu de réflexion inventé par le mathématicien français Edouard LUCAS. Ce jeu consiste à transférer n disques de diamètres différents d'une tour de départ A vers une tour

d'arrivée C en utilisant une tour intermédiaire B, et ceci en un minimum de coups, tout en respectant les règles suivantes :

- Un seul disque à la fois peut être déplacé ;
- Un disque doit toujours reposer sur un autre disque plus grand que lui ou sur un emplacement vide.

Cette dernière règle est respectée dans la configuration de départ.



Essayez de jouer ce jeu pour 1, 2, 3 disques. Combien de déplacement avez-vous fait ?

Si vous passez à 4 disques ; que remarquez-vous ?

Essayez d'écrire un algorithme récursif permettant de résoudre le problème avec N disques.